

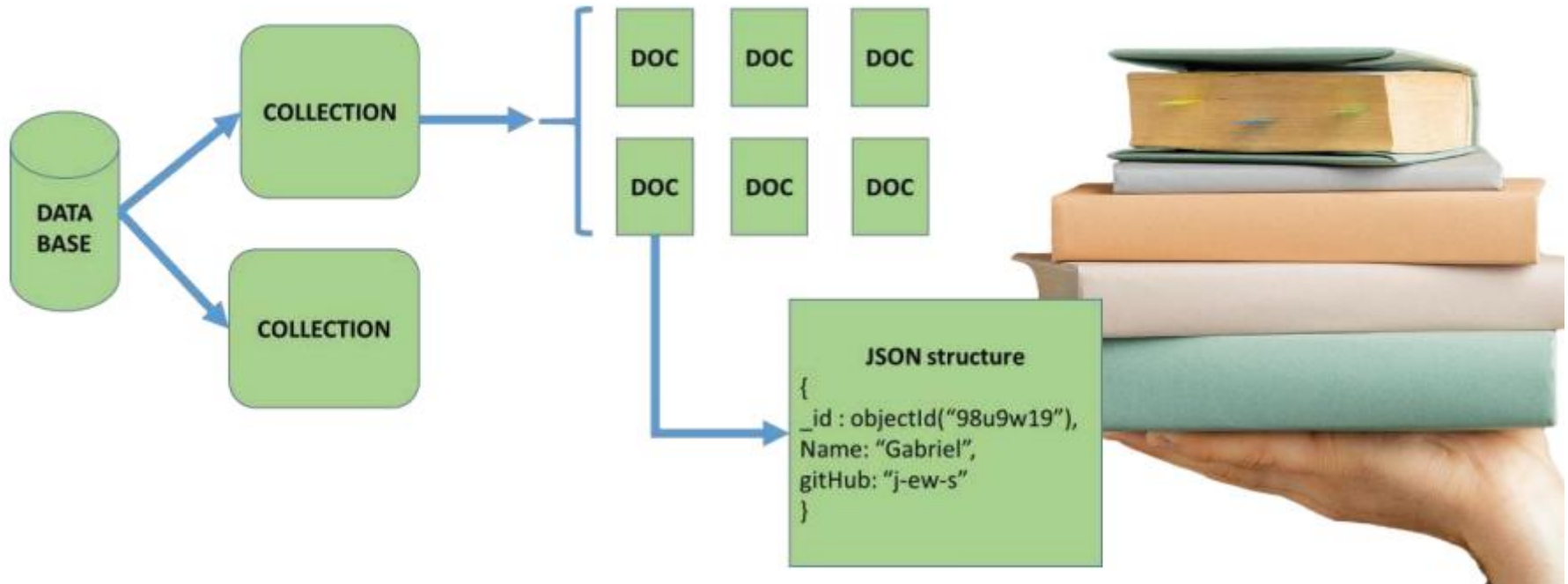
Banco de Dados NoSql

Professor Msc. Aparecido Vilela Junior
aparecido.vilela@unicesumar.edu.br

Armazenamento MongoDB

Relacional	MongoDB
Database / Banco / DB	Database / Banco / DB
Tabela	Collection / Coleção
Linhas	Document / Documento
Coluna	Field / campo
Index	Index
Joins	-
Primary Key / Chave	Primary Key / Chave

Estrutura MongoDB



Estrutura MongoDB

Cada documento, é uma informação do banco.

Eles são escritos em JSON.

O tamanho máximo de um arquivo JSON é de 16mb.



Armazenamento MongoDB

Banco Relacional		MongoDB
Banco de Dados	→	Banco de Dados
Tabela, Visão	→	<code>> db.usuario.findOne({idade:25})</code>
Linha	→	<code>{</code>
Coluna (Esquema Rígido)	→	<code> "_id" : ObjectId("1654e0bd42..."),</code>
Índice	→	<code> "primeiro" : "Bia",</code>
Junção	→	<code> "ultimo" : "Souza",</code>
Chave Estrangeira	→	<code> "idade" : 25,</code>
Partição	→	<code> "hobby" : [</code>
		<code> "Aviação",</code>
		<code> "Alpinismo"] ,</code>
		<code> "recomendações": {</code>
		<code> "cor": "Rosa",</code>
		<code> "Esporte": "Futebol"</code>
		<code>}</code>

Arrays

Atributo onde podemos incluir uma lista de valores.

```
"hobby" : [
    "Aviação",
    "Alpinismo" ] ,
```

Os conteúdos do array não possuem o formato chave, valor.

Documentos aninhados ou embutidos

Documento incluído
dentro de outro
documento.

```
"recomendações": {  
  "cor": "Rosa",  
  "Esporte": "Futebol"}  
}
```

Diminui os Joins
(vantagem de
desempenho).

Tipagem

Tipos dinâmicos

Todos os tipos suportados pelo formato BSON
são também suportados pelo MongoDB

String -> Armazenar dados

Inteiro -> Número inteiro de 32 e 64 bits.

Booleano -> Falso ou verdadeiro

Double -> Valores de pontos flutuantes

Tipagem

Min/Max Keys -> Usado para comparar um valor com os elementos BSON maior e menor.

Array -> Armazenar valores em uma chave

Timestamp -> Registrar data e hora. Muito útil para gravar quando um documento foi modificado ou acionado.

Object -> Esse tipo de dados é usado para documentos aninhados.

Tipagem

Null -> armazenando valores nulos.

Symbol -> forma idêntica a uma string,
reservados para idiomas que usam um tipo de
símbolo específico.

Date -> usado para armazenar a data ou hora
atual no formato de hora UNIX.

Object Id -> armazenar o ID do documento.

Tipagem

Binary Data -> armazenar valores binários

Code -> utilizado para armazenar o código JavaScript no documento.

Regular Expression -> armazenar expressões regulares.

Manipulação de dados

Inserção

`db.collection.insert(<document>)` -> Forma normal de incluir um novo documento na coleção (tabela).

`db.collection.save(<document>)`

O save se especificar o documento que está querendo salvar o `_id` e ele já existir ele irá atualizar o documento, se não existir ele insere.

`db.collection.update(<query>,<update>,{upsert:true})`

Passa um consulta, um filtro especificando os documentos que quer realizar a atualização No update, coloca os dados que quer substituir (valores novos), e o upsert -> true, se ele procurar e não encontrar nenhum documento, o mesmo será incluído.

Inserir dados

```
db.COLLECTION_NAME.insert(document)
```

No documento inserido, se não especificar o parâmetro `_id`, então MongoDB atribui um ObjectId exclusivo para esse documento.

`_id` é de 12 bytes número hexadecimal exclusivo para cada documento em uma coleção. 12 bytes são divididos da seguinte forma:

`_id`: ObjectId(4 bytes timestamp, 3 bytes machine id, 2 bytes process id, 3 bytes incrementer)

Manipulação de Dados

Para inserir vários documentos no MongoDB

`db.NomeColeção.insertMany([documento 1, documento 2,...])`

Para verificar a quantidade de documentos em uma coleção

`db.NomeColeção.count()`

Inserir dados - Múltiplos

```
>db.aula.insert([
  {_id:1,
    nome: 'NoSql',
    tags: ['mongodb', 'database', 'NoSQL'],
  },
  {
    _id:2,
    nome:'PO',
    tags: ['po', 'PL', 'PI'],
    comments: [
      { aluno:'aluno1',
        message: 'Muito fácil',
      }
    ]
  }
])
```

Manipulação de Dados

Atualização

`db.collection.update(<query>,<update>,<options>)`

`query` -> Predicado (filtros) -> Where (relacional)

`<update>` -> Campos a serem atualizados.

`<options>` - semelhante ao slide anterior.

Exclusão

`db.collection.remove(<query>, <justOne>)`

`<justOne>` ->

Manipulação de dados

Exclusão MongoDB

`db.NomeColeção.remove({nomeAtributo : valor do atributo})`

Se nenhum atributo for informado todos os dados da coleção são excluídos

`db.NomeColeção.remove({})`

Para excluir uma coleção inteira usamos o método `drop()`;

`db.NomeColeção.drop()`

Para excluir o banco de dados

`db.dropDatabase()`

Consultando Dados

Consulta

```
db.collection.find(<query>,<projection>)
```

Consultar os dados com os filtros e com os atributos que devem ser exibidos. Faz paginação, formatação, trabalha com estrutura de dados, como um cursor.

```
db.collection.findOne((<query>,<projection>)
```

Consultar com o retorno de somente um único documento

Exibir dados

```
>db.COLLECTION_NAME.find()
```

find() método irá exibir todos os documentos de uma forma não estruturada. Para exibir os resultados em uma maneira formatada, você pode usar pretty() método.

Sintaxe:

```
>db.aula.find().pretty()
```

Exibir dados

Operação	Sintaxe	Exemplo	RDBMS Equivalência
Equality	{<key>:<value>}	db.mycol.find({"by":"tutorials point"}).pretty()	where by = 'tutorials point'
Less Than	{<key>:{\$lt:<value>}}	db.mycol.find({"likes":{\$lt:50}}).pretty()	where likes < 50
Less Than Equals	{<key>:{\$lte:<value>}}	db.mycol.find({"likes":{\$lte:50}}).pretty()	where likes <= 50
Greater Than	{<key>:{\$gt:<value>}}	db.mycol.find({"likes":{\$gt:50}}).pretty()	where likes > 50
Greater Than Equals	{<key>:{\$gte:<value>}}	db.mycol.find({"likes":{\$gte:50}}).pretty()	where likes >= 50
Not Equals	{<key>:{\$ne:<value>}}	db.mycol.find({"likes":{\$ne:50}}).pretty()	where likes != 50

Exibir dados – AND / OR

No find() método se você passar várias chaves, separando-os por ", então MongoDB trata AND condição. Sintaxe básica AND é mostrado abaixo:

```
>db.aula.find({key1:value1, key2:value2}).pretty()
```

OR

```
>db.mycol.find(  
  {  
    $or: [  
      {key1: value1}, {key2:value2}  
    ]  
  }  
)
```

Consultas

Consultas

- Todas as consultas são feitas usando o método `db.collection.find()`
 - Recebe os critérios, projeção e retorna um cursor
- Todas as consultas operam em uma única coleção

```
db.users.find(  
  { age: { $gt: 18 } },  
  { name: 1, address: 1 }  
) .limit(5)
```

← coleção
← critérios da consulta
← projeção
← modificador do cursor

- Consulta em MongoDB:
- Em um banco relacional:

```
SELECT _id, name, address  
FROM users  
WHERE age > 18  
LIMIT 5
```

← projeção
← coleção
← critérios da consulta
← modificador do cursor

Consultas

SQL SELECT

MongoDB find()

```
SELECT *  
FROM users
```

```
db.users.find()
```

```
SELECT id,  
       user_id,  
       status  
FROM users
```

```
db.users.find(  
    { },  
    { user_id: 1, status: 1 }  
)
```

```
SELECT user_id, status  
FROM users
```

```
db.users.find(  
    { },  
    { user_id: 1, status: 1, _id: 0 }  
)
```

Where

DQL: Consultas - WHERE

SQL SELECT

```
SELECT *  
FROM users  
WHERE status = "A"
```

MongoDB find()

```
db.users.find(  
  { status: "A" }  
)
```

```
SELECT user_id, status  
FROM users  
WHERE status = "A"
```

```
db.users.find(  
  { status: "A" },  
  { user_id: 1, status: 1, _id: 0 }  
)
```

```
SELECT *  
FROM users  
WHERE status != "A"
```

```
db.users.find(  
  { status: { $ne: "A" } }  
)
```


WHERE

SQL SELECT

MongoDB find()

```
SELECT *  
FROM users  
WHERE status = "A"  
AND age = 50
```

```
db.users.find(  
  { status: "A",  
    age: 50 }  
)
```

```
SELECT *  
FROM users  
WHERE status = "A"  
OR age = 50
```

```
db.users.find(  
  { $or: [ { status: "A" } ,  
           { age: 50 } ] }  
)
```

```
SELECT *  
FROM users  
WHERE age > 25
```

```
db.users.find(  
  { age: { $gt: 25 } }  
)
```

LIKE

SQL SELECT

MongoDB find()

```
SELECT *  
FROM users  
WHERE age > 25  
AND age <= 50
```

```
db.users.find(  
  { age: { $gt: 25, $lte: 50 } }  
)
```

```
SELECT *  
FROM users  
WHERE user_id like "%bc%"
```

```
db.users.find( { user_id: /bc/ } )
```

```
SELECT *  
FROM users  
WHERE user_id like "bc%"
```

```
db.users.find( { user_id: /^bc/ } )
```

Ordenação

SQL SELECT

```
SELECT *  
FROM users  
WHERE status = "A"  
ORDER BY user_id ASC
```

MongoDB find()

```
db.users.find( { status: "A" } ).sort( { user_id: 1 } )
```

```
SELECT *  
FROM users  
WHERE status = "A"  
ORDER BY user_id DESC
```

```
db.users.find( { status: "A" } ).sort( { user_id: -1 } )
```

COUNT

SQL SELECT

```
SELECT COUNT(*)  
FROM users
```

```
SELECT COUNT(*)  
FROM users  
WHERE age > 30
```

```
SELECT DISTINCT(status)  
FROM users
```

MongoDB find()

```
db.users.count()
```

or

```
db.users.find().count()
```

```
db.users.count( { age: { $gt: 30 } } )
```

or

```
db.users.find( { age: { $gt: 30 } } ).count()
```

```
db.users.distinct( "status" )
```

Limit / Skip

SQL SELECT

```
SELECT *  
FROM users  
LIMIT 1
```

MongoDB find()

```
db.users.findOne()
```

or

```
db.users.find().limit(1)
```

```
SELECT *  
FROM users  
LIMIT 5  
SKIP 10
```

```
db.users.find().limit(5).skip(10)
```

SubDocumentos

- Considere uma coleção `inventory` com documentos semelhantes a:

```
{
  item: "Screwdriver",
  producer: {
    company: "Philips",
    address: "Amsterdan"
  },
  stock: [ { size: "S", qty: 25 }, { size: "M", qty: 50 } ],
  category: "tools"
}
```

- Para consultar campos de documentos embutidos, usamos a *dot notation*

- Um *match* exato de um "subdocumento":

```
db.inventory.find({ producer: {company: 'ABC', address: 'a' } })
```

- Campos individuais de um subdocumento:

```
db.inventory.find({ 'producer.company': 'ABC' });
```

- Elemento específico de um *array*:

```
db.inventory.find({ 'favorite_fruits.0': 'Apple' });
```

- [Referência](#) completa na documentação do MongoDB